Overloaded functions are extremely common in Elixir applications. They are regularly used for recursive functions and for changing behavior based on configuration. You'll see more examples of overloading throughout Part I.

Up next, we're going to take a look at a convenient shorthand to create anonymous functions.

Capturing Functions

Anonymous functions are created using the `fn` keyword. But there's another technique to create anonymous functions called function capturing. This technique lets you define an anonymous function with a short syntax. It's very popular when defining a one- or two-line function for use in modules like `Enum`.

We'll go over the syntax for function capturing as well as how it can be used on named functions. It's a quick bit of syntax, so let's dive right in!

Capturing Anonymous Functions

Captured functions begin with the `&` symbol. Inside the body of the function, `&1` is used to refer to the first argument. Here's an example:

```
$ iex
iex(1)> Enum.map([1, 2, 3], & &1 * 2)
[2, 4, 6]
```

The phrase `& &1 * 2` is a captured function. It's equivalent to `fn x -> x * 2 end`. For each enumeration of our list, the argument is passed into `&1` and then multiplied by 2.

Captured functions can reference multiple arguments. In the previous example, `Enum.map/2` calls a function that accepts a single argument. But a function like `Enum.reduce/2` calls a function that accepts two arguments. If we wanted to use this in an anonymous function, it would look like this:

```
iex(2)> Enum.reduce([1, 2, 3], 0, & &1 + &2)
6
```

It's fairly rare to see a captured function that uses `&2` or even `&3`. It just becomes too difficult to follow along with. In these cases, it's better to use `fn` to define the anonymous function.

The captured function syntax is a bit daunting at first, but it's a convenient way to define simple functions that span a single line. Many Elixir developers prefer this syntax for short iteration functions.